

Organización del Proyecto

TP_facultad/ (Carpeta raíz del proyecto)

- **src/**
 - **headers/**
 - CapaDatos.h
 - GridManager.h
 - NodoBateria.h
 - NodoConsumidor.h
 - NodoProsumidor.h
 - NodoRed.h
 - CapaDatos.cpp
 - GridManager.cpp
 - NodoBateria.cpp
 - NodoConsumidor.cpp
 - NodoProsumidor.cpp
 - NodoRed.cpp
 - main.cpp
- **util/**
 - config.ini (config base de datos)
 - Datos_ofertas_HH.csv
- **sql/**
 - schema.sql (Código SQL de la base de datos)
- **docs/**
 - Documentación
 - Diagrama de Clases UML
 - Diagrama Entidad-Relación

Estructura de Tablas

1. Tabla NODOS

Almacena a todos los participantes de la micro-red.

- **id_nodo** (INT / PRIMARY KEY): Identificador único del nodo.
- **ubicacion** (VARCHAR): Dirección o zona geográfica del nodo.
- **tipo** (VARCHAR): Define el rol en la red. Solo puede ser 'Consumidor', 'Prosumidor' o 'Bateria' (validado con una restricción CHECK).
- **saldo_cuenta** (NUMERIC): La cantidad de saldo disponible del nodo. Por defecto es 0 y tiene una restricción CHECK para evitar que caiga en valores negativos.
- **perfil_consumo** (VARCHAR): Define el tipo de consumo (solo Consumidores, ej: 'Residencial', 'Comercial', 'Industrial').

2. Tabla TRANSACCIONES

Registra cada compra-venta exitosa de energía procesada por el algoritmo de subasta.

- **id_transaccion** (INT / PRIMARY KEY): ID único autoincremental de la operación.
- **id_vendedor** (INT / FOREIGN KEY): El nodo que vende la energía (hace referencia a la tabla nodos).
- **id_comprador** (INT / FOREIGN KEY): El nodo que compra la energía (hace referencia a la tabla nodos).
- **kwh** (NUMERIC): Cantidad de energía transferida (debe ser mayor a 0).
- **precio_unitario** (NUMERIC): Precio acordado (precio de clearing) por cada kWh (debe ser mayor a 0).
- **fecha_transaccion** (TIMESTAMP): Fecha y hora exacta de la operación.

3. Tabla LECTURAS HISTÓRICAS

Funciona como el medidor de luz de cada nodo. Guarda el registro hora por hora de qué pasó con la energía de ese nodo.

- **id_lectura** (INT / PRIMARY KEY): ID autoincremental.
- **id_nodo** (INT / FOREIGN KEY): El nodo al que pertenece esta lectura (hace referencia a NODOS).
- **tick_hora** (TIMESTAMP): La hora del registro.
- **produccion_kwh** (NUMERIC): Cuánta energía produjo o vendió.
- **consumo_kwh** (NUMERIC): Cuánta energía consumió o compró.
- **excedente_neto** (NUMERIC): Balance final del tick (positivo si le sobró energía, negativo si demandó).

4. Tabla CONFIG_TARIFAS

Un catálogo de precios base por hora.

- **hora** (INT / PRIMARY KEY): La hora del día (del 0 al 23, validado con un CHECK).
- **precio_base_kwh** (NUMERIC): El precio de referencia de la energía para esa hora específica.

⚙️ Automatizaciones (Triggers y Procedimientos)

Automatización 1: El Disparador trg_validar_saldo

- **¿Cuándo actúa?:** Se ejecuta automáticamente **antes** (BEFORE) de intentar guardar una nueva fila en la tabla TRANSACCIONES.
- **¿Qué hace?:** Calcula el costo total de la operación ($\text{kwh} * \text{precio_unitario}$) y va a la tabla NODOS a mirar cuánta plata tiene el comprador .
- **Resultado:** Si el comprador no tiene plata suficiente, el trigger frena todo, aborta la transacción y lanza un error ("Saldo insuficiente...") . Si tiene saldo, deja que la inserción continúe.

Automatización 2: El Procedimiento actualizar_saldo_y_lecturas

- **¿Cuándo actúa?** Este procedimiento es llamado explícitamente en la base de datos **después** de confirmar que la transacción fue exitosa.
- **¿Qué hace?:** Recibe como parámetros el ID del nodo, cuánta energía se movió, a qué precio y si fue una "compra" o "venta". Con esos datos hace dos cosas fundamentales:
 - Actualiza el bolsillo:** Va a NODOS y le resta plata al comprador, o le suma plata al vendedor.
 - Registra la estadística:** Inserta una nueva fila en LECTURAS_HISTORICAS para dejar asentado que ese nodo produjo o consumió esa energía.

Clase NodoRed:

NodoRed es una clase abstracta, la cual es clase padre de (NodoConsumidor, NodoProsumidor, NodoBateria) porque estas clases tienen en común: id_nodo, ubicacion, saldo_cuenta y balanceEnergia.

Se optó por el acceso protected para los atributos comunes. Esto permite que las clases derivadas (NodoConsumidor, etc.) accedan directamente a ellos de forma eficiente, manteniendo estos datos ocultos y protegidos del resto del sistema exterior.

Las funciones **getTipo**, **imprimirDetalle** y **calcularExcedente** son virtuales puras porque obligan a las clases derivadas a implementar su propia lógica, permitiendo que el sistema trate a todos como NodoRed (polimorfismo) sin importar qué tipo de nodo son realmente en tiempo de ejecución. Al ser una clase abstracta con funciones virtuales, se requiere de un destructor virtual para asegurar que al destruir un nodo a través de un puntero a la clase base, la memoria de las clases hijas se libere correctamente y se eviten fugas de recursos (memory leaks).

El constructor de la clase no recibe el parámetro balanceEnergia debido a que al crear cualquier nodo en la red, este siempre inicia con un balance neutro de 0.0 unidades de energía.

En cuanto a las funciones que no son virtuales, consisten en métodos de acceso estándar (getters) para consultar datos inmutables de los nodos como el ID (getId), la ubicación (getUbicacion) y el saldo (getSaldo)

Clase NodoConsumidor:

NodoConsumidor es una clase hija de NodoRed y representa a los participantes de la red que únicamente consumen energía. En su sección privada posee un atributo perfil de tipo **PerfilConsumo**, el cual es un enum class (enumeración estructurada) por razones de eficiencia, seguridad de tipos y compatibilidad con estructuras switch, que determina de forma estricta si el perfil de consumo es Residencial, Comercial o Industrial , También se define un double **demanda_actual** que almacena temporalmente el consumo del tick actual para el cálculo de balance.

En su sección pública, se define el constructor que inicializa los datos llamando al constructor base de NodoRed y asignando el perfil correspondiente. Posee también una función **setDemandaActual** para cargar la demanda del nodo en el tick actual antes de ejecutar la subasta. Finalmente, implementa las funciones virtuales puras de la clase base: **getTipo** devuelve una representación textual del perfil del consumidor; **imprimirDetalle** muestra por consola el desglose de los datos del nodo; y **calcularExcedente** actualiza el atributo balanceEnergia en valor negativo (ya que representa una demanda/extracción de energía de la red) y lo retorna.

Clase NodoProsumidor:

NodoProsumidor es una clase hija de NodoRed y representa a los participantes de la red que consumen energía y también la generan para la venta. En su sección privada posee los atributos double produccion y consumo.

En su sección pública, se define el constructor que inicializa los datos llamando al constructor base de NodoRed y asignando los valores de producción y consumo correspondientes. Finalmente, implementa las funciones virtuales puras de la clase base: **getTipo** devuelve una representación de texto ("Prosumidor"); **imprimirDetalle** muestra por

consola el desglose de los datos del nodo (incluyendo su producción y consumo en kWh); y `calcularExcedente()` actualiza el atributo `balanceEnergia` con el balance neto actual (producción - consumo) y lo retorna (pudiendo ser positivo si hay excedente o negativo si hay déficit).

Clase NodoBateria:

Clase `NodoBateria` es una clase hija de `NodoRed`. Esta clase representa a la batería comunitaria o general del sistema, encargada de absorber los excedentes energéticos de los vendedores al final de cada tick y reinyectarlos al mercado en forma de oferta de venta en ticks posteriores. En su sección privada contiene el atributo `double carga_actual_kwh` que representa la cantidad de energía almacenada.

En su sección pública, se define el constructor que delega la inicialización básica al constructor de `NodoRed` y asigna explícitamente `carga_actual_kwh` en 0.0, ya que el dispositivo inicia descargado al acoplarse al sistema. Además, incluye getters y métodos específicos como **actualizarCarga** para modificar la energía disponible. Finalmente, implementa las funciones virtuales heredadas: **getTipo** retorna "Batería"; **imprimirDetalle** muestra por consola el ID, ubicación, saldo y carga actual de la batería en kWh; y **calcularExcedente** asigna el valor de la carga actual disponible a `balanceEnergia` (siempre positivo o cero) y lo retorna para ser inyectado como oferta de venta en el mercado.

Clase GridManager

Esta clase es el cerebro central de la red eléctrica. Solo puede existir una instancia de la misma, por lo que no puede duplicarse o crearse otra. Por ello, su constructor es privado porque al ponerla en la selección privada impide que en otra parte del programa se use el constructor. Para crearla se utiliza una función estática interna `instance` que se inicializa sólo la primera vez que se llama a la función y perdura durante toda la ejecución del programa. Por otra parte, también tiene definido el operador de asignación (`= delete`), esto evita que alguien clone el orquestador por error.

Otros datos declarados en privado son:

`Libro de compras(bidMap)`: está definido como un mapa cuya clave es un `double` (precio) y su valor es una cola (FIFO). Las claves de este mapa están ordenadas de mayor a menor, esto es crucial porque el mercado siempre le da prioridad a los compradores dispuestos a pagar más caro.

`Libro de ventas(askMap)`: está definido como un mapa igual. La única diferencia entre ellos es que el criterio de diferencia está definido por stock, es decir de menor a mayor. Es decir, da prioridad a los vendedores que ofrecen la energía más barata.

En su sección pública se encuentran definidas las diferentes funciones para gestionar la base de datos. `inyectarBaseDatos` recibe un puntero a `capaDatos`, es decir, no crea la base de datos como tal sino que se la inyecta desde `capaDatos` para separar tareas y lograr una correcta separación de responsabilidades. **setNodosActivos** obtiene la lista de todos los nodos activos en la base de datos, y `setHoraActual`, la hora actual de la simulación.

La función **cargarOfertasDelCSV** se encarga de leer el archivo de entrada y estructurar las ofertas iniciales del mercado eléctrico. Al comenzar, el sistema recorre los nodos activos buscando específicamente el de almacenamiento; para lograr acceder a sus métodos particulares, como la consulta de energía acumulada, se realiza una conversión segura de tipos mediante `dynamic_pointer_cast`, lo que permite inyectar automáticamente una orden prioritaria de venta si la batería dispone de carga. A continuación, el algoritmo procesa el archivo CSV, omitiendo el encabezado y empaquetando los datos en una estructura temporal donde los valores numéricos son redondeados para evitar errores de precisión. Para prevenir que fallos de tipeo del archivo .csv interrumpan el sistema, se aplica una limpieza sobre el campo de texto que indica el tipo de operación, eliminando espacios en blanco y convirtiendo los caracteres a minúsculas antes de compararlo con la palabra "compra" para generar un valor booleano. Finalmente, basándose en si este booleano resulta verdadero o falso, el sistema enruta la orden hacia el libro de compras (`bidMap`) o al de ventas (`askMap`), dejando la información correctamente clasificada para el inicio de la subasta.

La función **ejecutarSubasta** se encarga de cruzar las ofertas de compra y venta en base a la compatibilidad de precios: el precio máximo que el comprador está dispuesto a pagar debe ser mayor o igual al precio mínimo que el vendedor está dispuesto a cobrar.

Al iniciar, se define un vector de transacciones (`transacciones_del_tick`) para almacenar los cruces exitosos que luego se persistirán en la base de datos. También se realiza una clonación de los libros de compra y venta (`bidMap_clon` y `askMap_clon`). Esto permite trabajar sobre copias de seguridad en memoria por si la transacción falla en la base de datos y se debe ejecutar un ROLLBACK.

El algoritmo principal corre dentro de un bucle while que se ejecuta mientras existan ofertas tanto de compra como de venta. En cada iteración se evalúan los mejores precios de compra y venta (clave de los mapas). Si son compatibles, se toman las órdenes correspondientes ubicadas al frente de sus respectivas colas.

Se calcula la cantidad de energía a intercambiar como el mínimo entre lo que el comprador solicita y lo que el vendedor ofrece, y se promedian ambos precios para fijar el precio final de la transacción (precio de clearing). Con estos datos se instancia una estructura `TransaccionEnergia` guardando la fecha/hora actual, y se añade al vector de transacciones. Posteriormente, se actualizan los remanentes de energía de ambas órdenes. Si la energía de alguna orden llega a cero, ésta se remueve de la cola. Si una cola de precio se vacía por completo, se elimina la clave correspondiente del mapa clonado.

El bucle finaliza cuando no quedan más ofertas compatibles o una de las listas se vacía. A continuación, si aún quedan ofertas de venta sin concretar, se activa la batería comunitaria (la cual es buscada dinámicamente en el vector de nodos activos). La batería absorbe toda la energía sobrante de los vendedores al precio base de la tarifa de esa hora, generando las transacciones correspondientes.

Finalmente, si el vector de transacciones no está vacío, se envía a la base de datos mediante `persistirTransacciones()`.

El resultado se almacena en la variable éxito:

Si el COMMIT es exitoso se actualizan los libros originales de la memoria RAM con el estado de los clones procesados, se actualiza el estado de carga física de la batería en memoria y se descarta la demanda insatisfecha que no se pudo concretar.

Si falla (ROLLBACK): se descarta el mercado simulado en ese tick limpiando ambos libros para no arrastrar órdenes fallidas o inconsistentes a los siguientes ticks de la simulación.

Clase CapaDatos

Esta clase es la que se conecta a la base de datos. En la sección `private` define `SOCI`, la cual es una variable encapsulada de la librería, la cual es un tipo de dato que almacena y administra la conexión a la base de datos. Este objeto encapsula la conexión, maneja los estados (conectado/desconectado) y actúa como el punto de entrada principal para todas las operaciones SQL. También está **leerConfiguracion**, el cual es un método auxiliar que se encarga de analizar el archivo `config.ini` para construir de forma dinámica la conexión requerida por `postgresql`. Abre el archivo de configuración, si el archivo no existe o no se puede abrir interrumpe el flujo del sistema lanzando una excepción. Lee el archivo línea por línea y cualquier línea vacía o que comience con caracteres o cabeceras de sesión, las ignora. Utiliza una variable "ostringstream" para formatear la cadena final con los campos obligatorios que pide la conexión con `postgresql(dbname, user, password, host y port)`. Este método está en privado porque al resto del sistema no le interesa ni le corresponde saber como se lee el archivo de configuración, también para prevenir que otra clase lo invoque por error desde fuera de ella, protegiendo el funcionamiento interno de la base de datos.

En su sección pública se encuentra su constructor, que utiliza la función **leerConfiguracion** para extraer los datos del archivo `config`, y establece la conexión con la base de datos utilizando el método `sql.open(postgresql,datos_config)`, en el caso de que se pudo concretar la conexión tira un log "éxito". En caso de falla en la red o credenciales incorrectas, el constructor reporta un error crítico por consola sin provocar la caída abrupta de la simulación en memoria RAM.

El destructor de la clase garantiza la desconexión limpia del sistema con el motor de PostgreSQL, verifica de forma preventiva si la sesión se encuentra conectada mediante el método **sql.is_conenected**. si la conexión continúa activa, invoca el método para cerrar la sesión evitando fugas de sesiones abiertas.

La función **cargarNodos** realiza una consulta SQL a la tabla `public.nodos` para obtener la información de los participantes registrados y reconstruirlos dinámicamente en memoria RAM utilizando punteros inteligentes. Utilizamos punteros inteligentes para automatizar la liberación de forma automática de memoria mediante conteo de referencia interna. Cuando un vector de nodos activos se destruye o se limpia al finalizar la ejecución, el contador llega a cero y los destructores de los objetos en ram se disparan automáticamente, eliminando por completo fugas y riesgos de memoria. Dado que NodoRed es una clase abstracta, no es posible instanciar objetos directamente a ellas. El vector de punteros inteligentes a la clase base permite almacenar de forma polimórfica referencia de cualquier clase derivada. (NodoConsumidor, NodoProsumidor, NodoBateria), garantizando que al invocar los métodos virtuales se ejecute la lógica específica de cada uno.

Consulta: Prepara y ejecuta la consulta. El resultado se encapsula en una estructura iterable rowset que organiza datos en filas y columnas (como una tabla de base de datos) y permite recorrerlos uno por uno de forma secuencial.

Al recorrer cada fila, analiza el campo “tipo” para determinar el constructor específico a invocar:

Consumidor: Se utiliza el helper de SOCI para comprobar que el valor de “perfil_consumo” en la base de datos no sea nulo. Luego, convierte la representación de texto en un enum “PerfilConsumo” antes de ponerlo en el vector.

Prosumidor: Se inicializan los campos de generación e instancia un NodoProsumidor.

Bateria: Se instancia un NodoBateria el cual gestionará su propio almacenamiento energético.

La función **persistirTransacciones** es el núcleo de integridad de la economía de la aplicación, dado que garantiza la consistencia atómica entre la RAM y la base de datos.

Se instancia un objeto de transacción al iniciar la función. Cualquier consulta posterior se ejecutará en un entorno temporal aislado. Si el flujo se completa sin lanzar excepciones, se invoca **commit**. En caso de cualquier error, el bloque catch ejecuta **rollback**, deshaciendo todos los cambios de este tick y retornando false.

Conversión de Tiempos: Lo que hace es convertir la hora que le da el programa a una que sea compatible en la base de datos a través de la función.h que es el driver que SOCI puede traducir para luego colocarla en la columna TIME.

Protección contra Basura Decimal (Redondeo SQL): Aplican un casteo y redondeo estricto directamente en el motor PostgreSQL: ROUND(:parámetro::numeric, 3) para magnitudes de energía (kWh) y ROUND(:parámetro::numeric, 2) para montos de saldo y precio.

Invocación de Lógica de Base de Datos: Por cada transacción guardada, se ejecutan sentencias preparadas de llamadas a procedimientos almacenados (CALL public.actualizar_saldo_y_lecturas(...)) para actualizar los saldos del comprador y del vendedor y registrar sus lecturas de forma centralizada e histórica en la base de datos.

La función **obtenerPrecioBase** es utilizada para consultar de forma dinámica las tarifas del servicio en base a la hora de simulación actual.

Consulta la tabla public.config_tarifas el precio base a la hora correspondiente.

Política de Fallback: En caso de un error de base de datos, el método captura la excepción en un bloque try-catch, reporta la alerta por consola y retorna una tarifa predeterminada de 1.00 para asegurar que el simulador continúe ejecutándose sin detenerse abruptamente.

Ejecutar:

```
g++ -std=c++17 src/main.cpp src/GridManager.cpp src/CapaDatos.cpp src/NodoRed.cpp  
src/NodoConsumidor.cpp src/NodoProsumidor.cpp src/NodoBateria.cpp -o ecogrid  
-l/usr/include/postgresql -lsoci_core -lsoci_postgresql -lpq
```